



Manual Técnico de StokMaster

Guía para entender la estructura, conexiones, flujos y responsabilidades del código

Objetivo del manual

Este PDF explica cómo está dividido el proyecto, cómo se comunican las pantallas con los servicios, repositorios y SQLite, y qué función cumple cada bloque importante del código.

Dato	Valor
App	StokMaster
Versión app.json	2.1.1
Versión package.json	1.0.0
Expo SDK	54.0.8
React Native	0.81.5
Base de datos	SQLite local con expo-sqlite
Generado	2026-06-17 00:06

Nota: este manual se basa en el estado actual del código dentro de D:/Proyecto_StokMaster.

Contenido

- 1. Resumen ejecutivo
- 2. Tecnologías y configuración
- 3. Arquitectura general
- 4. Estructura de carpetas
- 5. Navegación y pantallas
- 6. Base de datos y relaciones
- 7. Servicios, repositorios y conexión entre capas
- 8. Flujos principales de la app
- 9. Reglas de negocio
- 10. Explicación por bloques de código
- 11. Reportes, PDF y respaldos
- 12. Tema, alertas y componentes globales
- 13. Cómo probar y mantener el proyecto
- 14. Anexo: inventario de archivos y funciones

Cómo leer este manual

No imprime literalmente cada línea del proyecto, porque eso haría el PDF poco práctico. En su lugar explica por capas, archivos, funciones y bloques: qué entra, qué procesa, qué guarda y con qué módulos se conecta.

1. Resumen ejecutivo

StokMaster es una aplicación móvil offline para administrar una pulpería. Su foco es registrar productos, controlar inventario, vender, escanear códigos de barras, generar alertas por bajo stock o vencimiento, crear reportes PDF y respaldar/restaurar los datos localmente.

La app está construida con Expo SDK 54.0.8 y Expo Router. Toda la información operativa vive en una base SQLite local llamada stokmaster.db, por eso el sistema puede funcionar sin internet.

Capas principales

Capa	Responsabilidad	Ejemplos
Pantallas	Reciben interacción del usuario y renderizan la interfaz.	src/app/inventario/registrar.jsx, src/app/ventas/nueva.jsx
Servicios	Aplican reglas de negocio y validaciones antes de tocar la base.	productService.js, salesService.js, backupService.js
Repositorios	Ejecutan SQL directo contra SQLite.	productRepository.js, salesRepository.js
Migración/DB	Crea tablas, columnas, índices y datos iniciales.	initDatabase.js, database.js
Soporte UI	Tema, alertas bonitas y navegación inferior.	AppThemeProvider.jsx, AppAlertProvider.jsx, AutoHideBottomNav.jsx

Mapa mental rápido

```

Usuario toca pantalla
-> src/app/... recibe el evento
  -> src/services/... valida reglas de negocio
    -> src/database/repositories/... ejecuta SQL
      -> SQLite local stokmaster.db guarda o consulta datos
    <- el resultado regresa al servicio
  <- la pantalla actualiza estado y muestra mensaje
  
```

2. Tecnologías y configuración

Elemento	Uso en el proyecto
Expo SDK 54.0.8	Base del proyecto y compatibilidad con Expo Go del teléfono indicado.
Expo Router	Rutas por archivos dentro de src/app. Cada archivo representa una pantalla o segmento.
React Native	Componentes móviles: View, Text, ScrollView, Pressable, FlatList, StyleSheet.
expo-sqlite	Base de datos local offline.
expo-camera	Escanear códigos de barras y control de flash.
expo-print	Generación de PDF desde HTML para reportes.
expo-file-system / legacy	Lectura/escritura de respaldos y PDF.
expo-sharing	Compartir/exportar archivos cuando el sistema lo permite.
expo-document-picker	Seleccionar respaldo JSON para restauración.
@expo/vector-icons	Iconos usados en botones, tarjetas y navegación.

Archivos de configuración

Archivo	Qué controla
package.json	Scripts de arranque y dependencias. Expo actual: 54.0.8.
app.json	Nombre, icono, splash, permisos, paquete Android y plugins. Versión visible: 2.1.1.
eas.json	Perfiles de build: development, preview APK y production.
tsconfig.json	Configuración TypeScript/base de Expo aunque el código principal es JS/JXSX.
AGENTS.md	Regla del proyecto: respetar documentación de Expo versión 54.0.8 antes de codificar.

Observación: package.json declara versión 1.0.0 y app.json declara 2.1.1. No rompe la app, pero conviene unificar criterios antes de una publicación formal.

3. Arquitectura general

La arquitectura está separada para que las pantallas no escriban SQL directamente. Las pantallas llaman servicios, los servicios aplican reglas y los repositorios son los únicos responsables del SQL.

Nivel	Ubicación	Qué contiene
1	src/app	Pantallas y rutas de Expo Router.
2	src/components/ui	Componentes globales reutilizables de navegación y alertas.
3	src/theme	Tema día/noche y colores globales.
4	src/services	Validaciones y reglas de negocio.
5	src/database/repositories	Consultas SQL específicas por entidad.
6	src/database/migrations	Creación, migración y ajustes de la base.
7	src/constants y src/utis	Catálogos de medida, moneda y fechas.

Flujo de dependencia recomendado

```
src/app/*
  importa servicios y componentes de UI
src/services/*
  importa repositorios, constantes y utilidades
src/database/repositories/*
  importa database.js y ejecuta SQL
src/database/connection/database.js
  abre stokmaster.db una sola vez
```

Regla práctica

Si una pantalla necesita guardar o consultar datos, debe hacerlo por medio de un servicio. Si un servicio necesita datos, debe usar un repositorio. Esta separación hace más fácil corregir errores sin tocar toda la app.

4. Estructura de carpetas

```
D:/Proyecto_StokMaster
  assets/images
    Logo, icono, splash y recursos visuales.
  src/app
```

```

    Rutas y pantallas principales de Expo Router.
src/app/inventario
    Listado, registro, detalle, edición, desactivados y movimientos.
src/app/ventas
    Panel de ventas, nueva venta, historial y detalle.
src/app/configuracion
    Respaldo, unidades, ayuda y acerca de.
src/components/ui
    Alertas globales y barra inferior auto-ocultable.
src/database
    Conexión, migraciones y repositorios SQLite.
src/services
    Reglas de negocio y orquestación de datos.
src/constants
    Catálogo de medidas y formato de moneda.
src/theme
    Proveedor de tema claro/oscuro.
src/utils
    Utilidades de fecha/hora de ventas.

```

La carpeta `src/app` define la navegación. La existencia de `src/app/inventario/[id].jsx`, por ejemplo, hace que Expo Router reconozca una pantalla dinámica para el detalle de producto.

5. Navegación y pantallas

Expo Router usa archivos como rutas. El layout raíz envuelve todas las pantallas con proveedores globales: tema, alertas, inicialización de SQLite y barra inferior.

Ruta	Archivo	Módulo	Función
/	src/app/index.jsx	Inicio	Resumen, acciones rápidas, drawer, tema y notificaciones.
/inventario	src/app/inventario/index.jsx	Inventario	Listado, filtros, búsqueda resaltada y acceso a detalles.
/inventario/registrar	src/app/inventario/registrar.jsx	Registrar producto	Formulario con categoría, unidad de medida, calendario, costo y precio.
/inventario/[id]	src/app/inventario/[id].jsx	Detalle/edición	Lectura y modificación de un producto existente.
/inventario/desactivados	src/app/inventario/desactivados.jsx	Desactivados	Reactivación, borrado definitivo y barra 0/10 dinámica.
/inventario/movimientos/[id]	src/app/inventario/movimientos/[id].jsx	Movimientos	Historial de cambios de stock por producto.
/ventas	src/app/ventas/index.jsx	Ventas	Entrada al módulo de ventas.
/ventas/nueva	src/app/ventas/nueva.jsx	Nueva venta	Carrito, escáner, cobro, stock y persistencia.
/ventas/historial	src/app/ventas/historial.jsx	Historial de ventas	Lista ventas y abre detalle.
/ventas/[id]	src/app/ventas/[id].jsx	Detalle de venta	Productos, totales, método de pago y cambio.
/reportes	src/app/reportes.jsx	Reportes	Día, semana, mes, año y exportación PDF.
/alertas	src/app/alertas.jsx	Alertas	Bajo stock y vencimiento.
/escaner	src/app/escaner.jsx	Escáner	Búsqueda por código y registro si no existe.

Ruta	Archivo	Módulo	Función
/categorias	src/app/categorias.jsx	Categorías	Alta y eliminación condicionada.
/configuracion	src/app/configuracion/index.jsx	Configuración	Menú de utilidades.
/configuracion/unidades	src/app/configuracion/unidades.jsx	Unidades	Referencia de medida.
/configuracion/respaldo	src/app/configuracion/respaldo.jsx	Respaldo	Exportar/importar JSON.
/configuracion/ayuda	src/app/configuracion/ayuda.jsx	Ayuda	Guía de uso.
/configuracion/acerca	src/app/configuracion/acerca.jsx	Acerca de	Información de la app.

Cómo se conectan las pantallas

- Inicio navega a inventario, escáner, ventas, reportes, alertas, categorías y configuración.
- Inventario abre detalle con `router.push(`/inventario/${item.id}`)`.
- Detalle abre movimientos con `/inventario/movimientos/[id]`.
- Nueva venta puede abrir escáner interno, agregar productos al carrito y luego redirigir al detalle de venta.
- Reportes usa rangos de fecha y exporta el PDF por medio de `reportExportService`.
- Configuración agrupa unidades, respaldo, ayuda y acerca de.

6. Base de datos y relaciones

La base se abre en `src/database/connection/database.js` con `SQLite.openDatabaseSync('stokmaster.db')`. La creación y migración ocurre en `initDatabase.js` cada vez que un servicio necesita datos.

Tabla	Campos principales	Responsabilidad
categorias	id, nombre, descripcion, activo, fecha_registro	Agrupa productos. Se puede eliminar solo si no quedan productos asociados no definitivos.
productos	id, nombre, codigo_barras, categoria_id, precio, precio_base, precio_compra, precio_compra_base, stock, stock_minimo, tipo_medida, unidad_base, presentacion_id, presentacion_nombre, cantidad_por_presentacion, fechas, activo	Inventario principal. Stock se guarda en unidad base para vender presentaciones o fracciones coherentes.
ventas	id, fecha, total, cantidad_productos, metodo_pago, recibido, cambio	Cabecera de cada venta. La fecha se guarda como UTC y se muestra en horario local.
detalle_ventas	venta_id, producto_id, producto_nombre, presentación, cantidades, precio_unitario, costo_unitario, costo_total, ganancia, subtotal	Detalle histórico de productos vendidos. Conserva nombre y precio aunque luego cambie el producto.
movimientos_inventario	producto_id, tipo, cantidad_anterior, cantidad_movida, cantidad_nueva, motivo, origen, referencia_id, responsable, fecha	Auditoría del stock: entradas, salidas por venta, ajustes y saldo inicial.
app_metadata	clave, valor	Preferencias o banderas internas, por ejemplo tema y semilla inicial.

Relaciones principales

```
categorias 1 --- N productos
productos 1 --- N detalle_ventas
ventas 1 --- N detalle_ventas
```

```

productos 1 --- N movimientos_inventario
productos 1 --- N alertas derivadas (no tabla fija; se calculan por stock/vencimiento)

```

Migraciones importantes

- ensureColumn revisa PRAGMA table_info y agrega columnas si faltan. Así una app ya instalada puede actualizarse sin perder datos.
- Se siembran categorías iniciales solo una vez usando app_metadata.
- Se recalculan precio_base, precio_compra_base y ganancia si existen datos viejos.
- Los productos eliminados definitivamente quedan con activo = -1 y pierden categoria_id, para permitir eliminar categorías sin asociaciones reales.
- Se crea un movimiento inicial para productos existentes: Saldo inicial al habilitar historial.

7. Servicios, repositorios y conexión entre capas

Servicio	Repositorio/soporte	Qué decide
productService.js	productRepository.js, categoryRepository.js, measurements.js	Valida producto, categoría, código único, medidas, precio/costo, stock y límite de desactivados.
salesService.js	salesRepository.js, productRepository.js, saleDateTime.js	Valida carrito, stock, pago, calcula costo/ganancia y fecha UTC.
categoryService.js	categoryRepository.js	Valida nombre, duplicados y bloqueo por productos asociados.
movementService.js	movementRepository.js	Valida producto y lista movimientos.
backupService.js	database.js, initDatabase.js	Exporta e importa tablas con validación de respaldo.
reportExportService.js	expo-print, file-system, sharing	Construye HTML de reporte y genera PDF seleccionando ubicación en Android.

Patrón de responsabilidad

- 1 La pantalla captura datos y estados visuales.
- 2 El servicio convierte esos datos a valores válidos para el negocio.
- 3 El repositorio ejecuta SQL con parámetros para evitar manipulación manual de cadenas.
- 4 El servicio devuelve resultado o lanza Error con mensaje claro.
- 5 La pantalla muestra Alert.alert, que AppAlertProvider convierte en diálogo con icono adecuado.

8. Flujos principales de la app

Registrar producto

- 1 La pantalla carga categorías activas desde categoryService.
- 2 El usuario completa nombre, categoría, precio de venta, precio de compra, stock, stock mínimo, unidad de medida y fecha opcional.
- 3 registrar.jsx valida campos vacíos y mueve el scroll al primer error.
- 4 productService.prepararProducto valida coherencia: unidad, presentación, categoría existente, código único y números válidos.

- 5 El stock se convierte a unidad base. Ejemplo: caja de 24 huevos se guarda como 24 unidades base.
- 6 `productRepository.insertarProducto` inserta en productos y registra movimiento inicial.

Editar producto

- 1 La ruta `/inventario/[id]` carga el producto por id.
- 2 El formulario reutiliza la lógica de medida y validación.
- 3 Al guardar, `productService.editarProducto` prepara datos y `productRepository.actualizarProducto` compara stock anterior contra nuevo.
- 4 Si cambia el stock, se agrega un movimiento de inventario tipo ajuste.

Vender

- 1 `ventas/nueva.jsx` carga productos activos y permite agregarlos por unidad base o por presentación.
- 2 Si el producto solo se vende por unidad, solo se muestra el botón de unidad.
- 3 El carrito acumula cantidades por producto y valida que no supere el stock.
- 4 `salesService.crearVenta` vuelve a validar stock en base de datos antes de guardar.
- 5 `salesRepository.insertarVenta` usa transacción: inserta venta, detalle, descuenta stock y registra movimiento salida.

Escanear

- 1 `escaner.jsx` y `ventas/nueva.jsx` usan `expo-camera`.
- 2 El flash puede encenderse/apagarse, se apaga al abrir registro y se restaura al volver a escanear si correspondía.
- 3 El código se busca con `productService.buscarProductoPorCodigo`.
- 4 Si existe, abre detalle o agrega al carrito. Si no existe, ofrece registrar producto con el código prellenado.

Reportar

- 1 `reportes.jsx` calcula rangos diarios, semanales, mensuales y anuales.
- 2 `salesService.obtenerReporteVentasPeriodo` valida fechas y delega a `salesRepository`.
- 3 `salesRepository` agrupa por fecha local usando utilidades de `saleDateTime`.
- 4 `reportExportService` arma HTML con resumen tipo ASCII visual, logo, detalle de productos por venta y lo exporta a PDF.

Respaldo/restaurar

- 1 `backupService.crearRespaldo` lee tablas principales y crea `respaldoStok_Master_fecha.json`.
- 2 En Android usa `Storage Access Framework` para que el usuario elija carpeta.
- 3 `restaurarRespaldo` valida app, versión y tablas requeridas.
- 4 La restauración usa transacción: borra tablas, inserta datos y ejecuta `initDatabase` para migraciones pendientes.

9. Reglas de negocio

Regla	Explicación
Productos activos	Aparecen en inventario, ventas, escáner y reportes según corresponda.
Productos desactivados	Dejan de aparecer en inventario normal. Pueden reactivarse o borrarse definitivamente.
Límite de desactivados	Máximo 10 productos guardados en el apartado de desactivados.
Borrado definitivo	Marca activo = -1 y libera categoría asociada para que la categoría pueda eliminarse si ya no tiene productos reales.
Categorías	Solo se eliminan si no tienen productos asociados que sigan activos o desactivados recuperables.
Unidades coherentes	Los tipos definen presentaciones permitidas: unidad, peso y volumen. Evita ventas ilógicas como litros de huevo.
Unidad mínima	Peso: 1/2 libra como mínima. Volumen: 1/2 litro como mínima. Unidad: pieza/base.
Precio de compra	Permite calcular ganancia estimada real en ventas y reportes.
Stock	Se guarda como unidad base para descontar fracciones/presentaciones de manera consistente.
Fechas de venta	Se guardan en UTC y se convierten a fecha local para mostrar y agrupar reportes.
Alertas	Se derivan de bajo stock y fecha de vencimiento próxima.
Respallos	El respaldo incluye tablas operativas y movimientos para conservar historial.

10. Explicación por bloques de código

Esta sección explica cómo leer las partes del código sin perderse entre archivos. La idea es ubicar responsabilidades, no memorizar cada línea.

Anatomía de una pantalla React Native

Bloque	Para qué sirve
Imports	Traen React, componentes de React Native, iconos, servicios, constantes, tema y router.
Estados useState	Guardan información temporal de la pantalla: campos de formulario, loading, carrito, filtros, errores.
useCallback/useEffect	Cargan datos y evitan recrear funciones innecesariamente.
useMemo	Calcula valores derivados como totales, filtros, resúmenes o colores.
Handlers	Funciones llamadas por botones: guardar, eliminar, escanear, confirmar, exportar.
Validación	Antes de llamar servicios, marca errores visuales y dirige al campo faltante.
Render JSX	Define lo visible: ScrollView, tarjetas, botones, listas, modales, DateTimePicker, CameraView.
StyleSheet	Centraliza tamaños, colores base y distribución visual.

Bloques clave por archivo

Archivo	Bloques importantes	Lectura recomendada
_layout.jsx	initDatabase, AppThemeProvider, AppAlertProvider, AutoHideBottomNav, Stack	Empieza aquí para entender cómo se envuelve toda la app.

Archivo	Bloques importantes	Lectura recomendada
index.jsx	abrirMenu, cerrarMenu, navegarDesdeMenu, ActionCard, SummaryCard	Explica el dashboard, drawer lateral y accesos rápidos.
inventario/regar.js	cargarCategorías, seleccionarFecha, validarFormulario, guardarProducto	Muestra cómo los campos del producto llegan al servicio.
inventario/[id].jsx	cargarProducto, validarFormulario, guardarCambios, confirmarEliminación	Es el equivalente de editar/desactivar con datos existentes.
ventas/nueva.jsx	agregarProducto, actualizarCantidadCarrito, abrirEscaner, manejarCodigoEscaneado, confirmarVenta, guardarVenta	Es el flujo más importante: carrito, stock, escáner y venta.
reportes.jsx	obtenerRangoPeriodo, exportarPdf, SummaryCard, EmptyState	Conecta filtros de fecha con consulta y PDF.
initDatabase.js	ensureColumn, CREATE TABLE, seed inicial, UPDATE de migración	Define la verdad de la base de datos.
productService.js	prepararProducto, crearProducto, editarProducto, eliminarProducto	Reglas centrales de inventario.
salesService.js	crearVenta, obtenerReporteVentasPeriodo	Reglas centrales de ventas y ganancia.
reportExportService.js	crearResumen*, crearHtmlReporte, exportarReporteVentasPdf	Generación final del PDF de reportes.
AppAlertProvider.jsx	getAlertVariant, Alert.alert override, modal custom	Convierte mensajes en diálogos con check, X o warning.

Ejemplo: producto nuevo

```

regar.js
  validarFormulario()
    -> revisa nombre, categoría, precio, costo, stock y vencimiento
  guardarProducto()
    -> crearProducto(payload)
productService.js
  prepararProducto(payload)
    -> calcula precio_base, precio_compra_base, stock base
productRepository.js
  insertarProducto(producto)
    -> INSERT INTO productos
    -> INSERT INTO movimientos_inventario

```

Ejemplo: venta

```

ventas/nueva.js
  agregarProducto(producto, modo)
    -> agrega presentación o unidad base al carrito
  confirmarVenta()
    -> pide confirmación visual tipo peligro
  guardarVenta()
    -> crearVenta({ carrito, metodo_pago, recibido })
salesService.js
  crearVenta()
    -> acumula cantidades por producto
    -> valida stock real en base
    -> calcula costo_total y ganancia
salesRepository.js
  insertarVenta()
    -> BEGIN TRANSACTION
    -> INSERT ventas + detalle_ventas
    -> UPDATE productos SET stock = stock - cantidad
    -> INSERT movimientos_inventario tipo salida
    -> COMMIT

```

11. Reportes, PDF y respaldos

Reportes

- El usuario elige periodo: Día, Semana, Mes o Año.
- reportes.jsx calcula fechaInicio y fechaFin.
- salesRepository filtra por fecha local, no solo por texto crudo UTC.
- El reporte incluye total, cantidad de ventas, productos vendidos, ganancia estimada, productos más/menos vendidos y detalle por venta.
- reportExportService usa expo-print para crear el PDF y Storage Access Framework en Android para elegir ubicación.

Respaldos

- El respaldo se llama respaldoStok_Master_YYYY-MM-DD_HH-MM.json.
- Incluye categorías, productos, ventas, detalle_ventas y movimientos_inventario.
- La restauración valida que el archivo sea de StokMaster y que tenga las tablas requeridas.
- Después de restaurar, initDatabase vuelve a garantizar columnas y compatibilidad.

12. Tema, alertas y componentes globales

Componente	Función
AppThemeProvider	Guarda modo light/dark en app_metadata y entrega colors a las pantallas.
useAppTheme	Hook que cada pantalla usa para pintar fondos, textos, tarjetas e inputs.
AppAlertProvider	Intercepta Alert.alert y decide si mostrar check verde, X roja o warning amarillo.
AutoHideBottomNav	Muestra la barra inferior, la oculta tras 2 segundos y reserva espacio para no tapar contenido.

13. Cómo probar y mantener el proyecto

Pruebas manuales recomendadas antes de una versión estable

- 1 Ejecutar `npx expo start --clear` y abrir en Expo Go compatible con SDK 54.
- 2 Registrar producto con categoría, precio de venta, precio de compra, unidad de medida y vencimiento.
- 3 Editar el producto y verificar que el historial de movimientos muestre ajuste si cambió stock.
- 4 Escanear código existente y no existente. Confirmar flash y navegación a registro.
- 5 Hacer venta por unidad y por presentación, validar descuento de stock y detalle de venta.
- 6 Generar reporte diario/semanal/mensual/anual y confirmar que aparezcan ventas del periodo.
- 7 Exportar PDF y verificar nombre Reporte_StockMaster_fecha.
- 8 Crear respaldo, restaurar respaldo y comprobar conteos.
- 9 Cambiar tema nocturno y revisar formularios, ventas, reportes, respaldo y productos desactivados.
- 10 Eliminar categorías solo cuando no tengan productos asociados recuperables.

Comandos útiles

```
npm install
npx expo start --clear
npx expo start --lan
npx expo start --tunnel
git status --short
git diff
```

Mantenimiento

Cuando agregues una regla de negocio, ponla en services. Cuando agregues SQL, ponlo en repositories o migrations. Cuando agregues una pantalla, ponla en src/app y navega con router.push.

14. Anexo: inventario de archivos y funciones

Este anexo se generó leyendo los archivos actuales de src. Sirve como mapa rápido para ubicar responsabilidades y puntos de entrada.

Archivo	Líneas	Responsabilidad	Funciones/bloques detectados
src/app/_layout.jsx	85	Raíz de Expo Router. Inicializa la base de datos, aplica tema, alertas globales, barra inferior y definición del Stack.	L9: export default function Layout() { L21: function ThemedLayout() {
src/app/alertas.jsx	437	Alertas de bajo stock y vencimiento. Calcula conteos y muestra prioridad visual.	L24: export default function AlertsScreen() { L143: function SummaryBox({ color, label, value }) { L154: function AlertCard({ alerta }) { L212: function getDaysText(days) { L230: function getUrgencyColor(priority, type) {
src/app/categorias.jsx	364	Gestión de categorías. Permite crear y eliminar si no hay productos no eliminados permanentemente asociados.	L22: export default function CategoriesScreen() { L49: const guardarCategoria = async () => { L69: const confirmarEliminarCategoria = (categoria) => {
src/app/configuracion/acerca.jsx	49	Información de versión, nombre y objetivo de StokMaster.	L5: export default function AboutScreen() {
src/app/configuracion/ayuda.jsx	189	Guía de uso dentro de la app.	L74: export default function HelpScreen() {
src/app/configuracion/index.jsx	158	Menú de configuración. Centraliza accesos a unidades, respaldo, ayuda y acerca de.	L47: export default function SettingsScreen() {
src/app/configuracion/respaldo.jsx	353	Pantalla de respaldo/restauración. Pide ubicación, confirma restauración y muestra resumen.	L16: export default function BackupScreen() { L21: const manejarCrearRespaldo = async () => { L47: const confirmarRestauracion = () => { L62: const manejarRestaurarRespaldo = async () => { L196: function SummaryItem({ label, value }) {
src/app/configuracion/unidades.jsx	369	Explica tipos de medida y presentaciones: unidad, peso y volumen.	L54: export default function UnitsScreen() {
src/app/escaner.jsx	279	Escáner independiente de códigos. Busca producto por código, enciende/apaga flash y permite registrar si no existe.	L32: export default function ScannerScreen() { L50: const manejarCodigo = async ({ data }) => {
src/app/index.jsx	690	Pantalla principal. Resume inventario, alertas, accesos rápidos, drawer lateral y botones superiores de tema/notificaciones.	L109: export default function HomeScreen() { L142: const abrirMenu = () => { L152: const cerrarMenu = (despuesDeCerrar) => { L163: const navegarDesdeMenu = (route) => { L388: function ActionCard({ icon, route, title, subtitle }) { L405: function SummaryCard({ backgroundColor, icon, label, value }) {

Archivo	Líneas	Responsabilidad	Funciones/bloques detectados
src/app/inventario/[id].jsx	999	Detalle y edición de producto. Permite cambiar datos, fechas, unidad de medida, desactivar y abrir movimientos.	L34: export default function ProductDetailScreen() { L98: const formatearFecha = (date) => { L110: const limpiarError = (field) => { L122: const actualizarCampo = (field, value) => { L168: const seleccionarTipoMedida = (typeid) => { L178: const registrarPosicion = (field) => (event) => { L182: const irAlCampo = (field) => { L193: const seleccionarFecha = (event, selectedDate) => { L205: const validarFormulario = () => { ... +3 más
src/app/inventario/desactivados.jsx	520	Productos desactivados. Controla el límite de 10, permite reactivar y borrar definitivamente.	L24: const getCapacityColor = (total) => { L36: export default function DisabledProductsScreen() { L69: const confirmarReactivar = (producto) => { L91: const confirmarBorrado = (producto) => { L284: function InfoRow({ label, value }) {
src/app/inventario/index.jsx	647	Listado de productos activos. Incluye filtros por estado, categoría, nombre, código y precio con resaltado de coincidencias.	L31: export default function InventarioScreen() { L78: const limpiarFiltros = () => { L318: function coincideConBusqueda(producto, term, tipoBusqueda) { L348: function HighlightedText({ highlight, style, text }) { L393: function productoVencePronto(fechaVencimiento) { L406: function StatusBadge({ color, text }) {
src/app/inventario/movimientos/[id].jsx	351	Historial de stock por producto. Muestra entradas, salidas, ajustes, venta relacionada y responsable.	L38: export default function ProductMovementsScreen() { L128: function MovementCard({ item, unidadBase }) { L184: function StockValue({ label, value }) { L195: function InfoRow({ label, value }) {
src/app/inventario/registrar.jsx	749	Alta de producto. Valida campos, abre calendario, carga categorías, calcula precio base/costo base y guarda en SQLite.	L29: export default function RegistrarProductoScreen() { L129: const formatearFecha = (date) => { L141: const seleccionarFecha = (event, selectedDate) => { L153: const seleccionarTipoMedida = (typeid) => { L160: const registrarPosicion = (field) => (event) => { L164: const irAlCampo = (field) => { L175: const limpiarError = (field) => { L187: const validarFormulario = () => { L250: const guardarProducto = async () => {
src/app/reportes.jsx	599	Reportes por día, semana, mes y año. Calcula rangos, consulta ventas y exporta PDF.	L30: export default function ReportsScreen() { L78: const seleccionarFecha = (event, selectedDate) => { L90: const exportarPdf = async () => { L286: function obtenerRangoPeriodo(date, periodo) { L321: function addDays(date, days) { L328: function formatDate(date) { L336: function SummaryCard({ label, value }) { L351: function SectionTitle({ title }) { L359: function EmptyState({ text }) {

Archivo	Líneas	Responsabilidad	Funciones/bloques detectados
src/app/ventas/[id].jsx	344	Detalle de venta. Muestra productos vendidos, pago, cambio, total y ganancia cuando existe en detalle.	L17: export default function SaleDetailScreen() { L144: function InfoRow({ label, value }) { L153: function SummaryItem({ icon, label, value }) { L163: function PaymentBadge({ method }) { L176: function capitalize(value) {
src/app/ventas/historial.jsx	291	Historial de ventas. Lista ventas registradas y conecta con detalle.	L18: export default function SalesHistoryScreen() { L144: function PaymentBadge({ method }) {
src/app/ventas/index.jsx	144	Módulo de ventas. Resume ventas y enlaza a nueva venta e historial.	L10: export default function SalesScreen() { L71: function SummaryCard({ label, value }) {
src/app/ventas/nueva.jsx	1013	Nueva venta. Arma carrito, escanea productos, valida stock, permite vender por unidad/presentación y guarda la venta.	L40: const tienePresentacionAdicional = (producto) => L44: export default function NewSaleScreen() { L90: const cantidadEnCarrito = (productold, current = carrito, exceptKey = null) => L96: const agregarProducto = (producto, modo) => { L152: const actualizarCantidadCarrito = (key, cantidadTexto) => { L196: const quitarProducto = (key) => { L211: const eliminarProductoCarrito = (key) => { L215: const stockRestante = (item) => L218: const abrirEscaner = async () => { ... +3 más
src/components/ui/AppAlertProvider.jsx	303	Reemplaza Alert.alert con diálogos personalizados de éxito, error y peligro/warning.	L45: const normalize = (value = "") => L53: const getAlertVariant = (title = "") => { L98: export default function AppAlertProvider({ children }) { L130: const closeDialog = () => { L134: const handleButtonPress = (button) => {
src/components/ui/AutoHideBottomNav.jsx	238	Barra inferior que se oculta tras inactividad, reaparece al tocar/scroll y reserva espacio para no tapar contenido.	L40: const getActiveSection = (pathname) => { L60: const shouldShowNav = (pathname) => !pathname.startsWith("/escaner"); L62: export default function AutoHideBottomNav({ children }) { L139: const handleNavPress = (route) => {
src/constants/measurements.js	57	Catálogo de tipos de medida, presentaciones permitidas, moneda y formato de cantidades.	L1: export const MEASUREMENT_TYPES = [L22: export const PRESENTATION_OPTIONS = { L42: export const getMeasurementType = (typeid) => L45: export const getPresentationOptions = (typeid) => L48: export const getPresentation = (typeid, presentationId) => L53: export const formatCurrency = (value) => L56: export const formatBaseQuantity = (quantity, baseLabel) =>

Archivo	Líneas	Responsabilidad	Funciones/bloques detectados
src/database/connection/database.js	5	Abre la base local SQLite stokmaster.db con expo-sqlite.	Sin funciones exportadas detectadas por regex.
src/database/migrations/initDatabase.js	256	Crea tablas, índices, columnas faltantes, datos iniciales y movimientos iniciales.	L3: const ensureColumn = async (tableName, columnName, definition) => { L22: export const initDatabase = async () => {
src/database/repositories/categoryRepository.js	101	SQL directo para categorías: insertar, listar, contar asociaciones, eliminar y buscar por id.	L3: export const insertarCategoria = async ({ descripcion = '', nombre }) => { L18: export const obtenerCategorias = async () => { L49: export const contarProductosAsignadosCategoria = async (id) => { L66: export const eliminarCategoriaPorId = async (id) => { L87: export const obtenerCategoriaPorId = async (id) => {
src/database/repositories/movementRepository.js	27	Consulta movimientos de inventario por producto.	L3: export const obtenerMovimientosPorProducto = async (productoid) => {
src/database/repositories/productRepository.js	508	SQL directo para productos: CRUD lógico, búsqueda por código, alertas, resumen y movimientos de stock.	L4: export const insertarProducto = async (producto) => { L99: export const obtenerProductos = async () => { L118: export const obtenerProductosDesactivados = async () => { L137: export const contarProductosDesactivados = async () => { L150: export const obtenerProductoPorId = async (id) => { L171: export const obtenerProductoPorCodigo = async (codigo) => { L192: export const actualizarProducto = async (id, producto) => { L300: export const desactivarProducto = async (id) => { L314: export const reactivarProducto = async (id) => { ... +4 más
src/database/repositories/salesRepository.js	279	SQL directo para ventas: transacción, detalle, descuento de stock, reportes y resúmenes.	L3: export const insertarVenta = async ({ L124: export const obtenerVentas = async () => { L139: export const obtenerVentaPorId = async (id) => { L165: export const obtenerResumenVentas = async () => { L187: export const obtenerReporteVentasPorFecha = async (fecha) => { L191: export const obtenerReporteVentasPorRango = async (fechaInicio, fechaFin) => {
src/services/backupService.js	192	Exporta/importa JSON con tablas principales y movimientos usando Storage Access Framework.	L24: const formatBackupDate = () => { L35: const getBackupFileName = () => `respaldoStok_Master_\${formatBackupDate()}.json`; L37: const insertRows = async (tableName, rows) => { L50: export const crearRespaldo = async () => { L121: export const restaurarRespaldo = async () => {

Archivo	Líneas	Responsabilidad	Funciones/bloques detectados
src/services/categoryService.js	55	Reglas de categoría: validación de nombre duplicado y bloqueo si tiene productos asociados activos/desactivados.	L9: export const listarCategorias = async () => { L14: export const crearCategoría = async (categoría) => { L31: export const eliminarCategoría = async (id) => {
src/services/movementService.js	14	Valida id y delega la consulta del historial de movimientos.	L4: export const listarMovimientosProducto = async (id) => {
src/services/productService.js	249	Reglas de negocio de producto: validaciones, medidas, precio base, límite de desactivados y datos de inicio.	L25: const prepararProducto = async (producto) => { L123: export const crearProducto = async (producto) => { L131: export const obtenerProducto = async (id) => { L149: export const editarProducto = async (id, producto) => { L163: export const eliminarProducto = async (id) => { L183: export const listarProductos = async () => { L188: export const buscarProductoPorCodigo = async (codigo) => { L200: export const listarProductosDesactivados = async () => { L205: export const restaurarProducto = async (id) => { ... +3 más
src/services/reportExportService.js	1049	Construye HTML de reportes y genera PDF con nombre, logo, rangos y detalle de venta.	L15: const escapeHtml = (value) => L23: const formatDate = (date = new Date()) => L30: const formatFileDate = (date = new Date()) => L37: const getReportFileName = () => `Reporte_StockMaster_\${formatFileDate()}.pdf`; L39: const formatDisplayDate = (value) => { L78: const parseLocalDate = (value) => new Date(`\${value}T12:00:00`); L80: const formatWeeklyRange = ({ fechaFin, fechaInicio }) => { L95: const formatMonthLabel = ({ fechaInicio }) => { L101: const formatYearLabel = ({ fechaInicio }) => { ... +27 más
src/services/salesService.js	160	Reglas de negocio de venta: carrito, stock, costos, ganancia, fecha UTC y persistencia.	L13: export const crearVenta = async ({ carrito, metodo_pago, recibido }) => { L107: export const listarVentas = async () => { L112: export const obtenerVenta = async (id) => { L130: export const obtenerDatosVentas = async () => { L135: export const obtenerReporteVentas = async (fecha) => { L145: export const obtenerReporteVentasPeriodo = async ({ fechaFin, fechaInicio }) => {
src/theme/AppThemeProvider.jsx	109	Tema día/noche. Lee y guarda preferencia en app_metadata y expone useAppTheme.	L42: export function AppThemeProvider({ children }) { L48: const cargarTema = async () => { L73: const toggleTheme = async () => { L101: export const useAppTheme = () => {

Archivo	Líneas	Responsabilidad	Funciones/bloques detectados
src/utls/saleDateTime.js	52	Normaliza fechas de ventas: guarda en UTC y muestra/agrupa en fecha local.	L1: export function formatUtcSaleTimestamp(date = new Date()) { L12: export function parseSaleDateTime(value) { L32: export function formatLocalDateKey(date) { L39: export function formatLocalSaleDateTime(value) {

Checklist de versión estable

- Dependencias alineadas a Expo SDK 54.0.8.
- Base SQLite inicializa sin errores y soporta migraciones de datos previos.
- Flujos mínimos: registrar, editar, escanear, vender, reportar, respaldar/restaurar.
- Tema claro/oscuro aplicado a pantallas principales.
- PDF de reporte incluye logo, nombre, fecha, resumen y detalle por venta.
- Historial de movimientos registra saldo inicial, ajustes y salidas por venta.